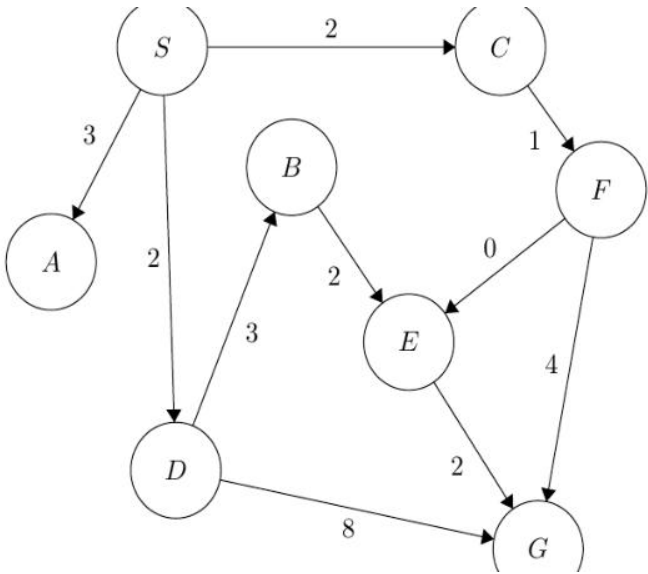


BÀI TẬP TÌM KIẾM TRONG AI

Họ và tên: Lư Hồng Phúc MSSV / Lớp: 3123410274-Lớp Chiều thứ 7

Bài 1. Các thuật toán tìm kiếm



1.1 và 1.2. BFS, DFS

Thuật toán	Explored set (theo thứ tự)	Path	Total cost
BFS	S, A, C, D, B, F	S → D → G	2 + 8 = 10
DFS	S, A, C, F, E	S → C → F → E → G	2 + 1 + 0 + 2 = 5

Quy ước: kiểm tra đích khi lấy nút khỏi frontier; nếu đúng đích thì dừng và chưa đưa nút đó vào explored set. Vì vậy G có trong đường đi nhưng không xuất hiện ở cột explored set.

Thứ ngày

BFS: (hư liên thông, duyệt rộng) là duyệt hết các nút ở độ sâu 1, 2, 3, ...
 Duyệt hàng chờ Q. (FIFO)

Explore set: S, A, C, D, F, B

Path: S → D → G

Fee: 2 + 8 = 10

No. ngày

Thứ

- 1 S : Q = [A, C, D]
- 2 A : Q = [C, D]
- 3 C : Q = [D, F]
- 4 D : Q = [F, B, G]
- 5 F : Q = [B, G, E]
- 6 B : Q = [G, E]
- 7 G : Q = [E]

No. ngày

Thứ

DFS: là duyệt liên tục: đi sâu hết 1 nhánh rồi mới quay lại xét nhánh khác

- 1 S : A, C, D
- 2 A : C, D
- 3 C : F, D
- 4 F : F, G, D
- 5 F : G, D
- 6 G : đích.

No. ngày

Thứ

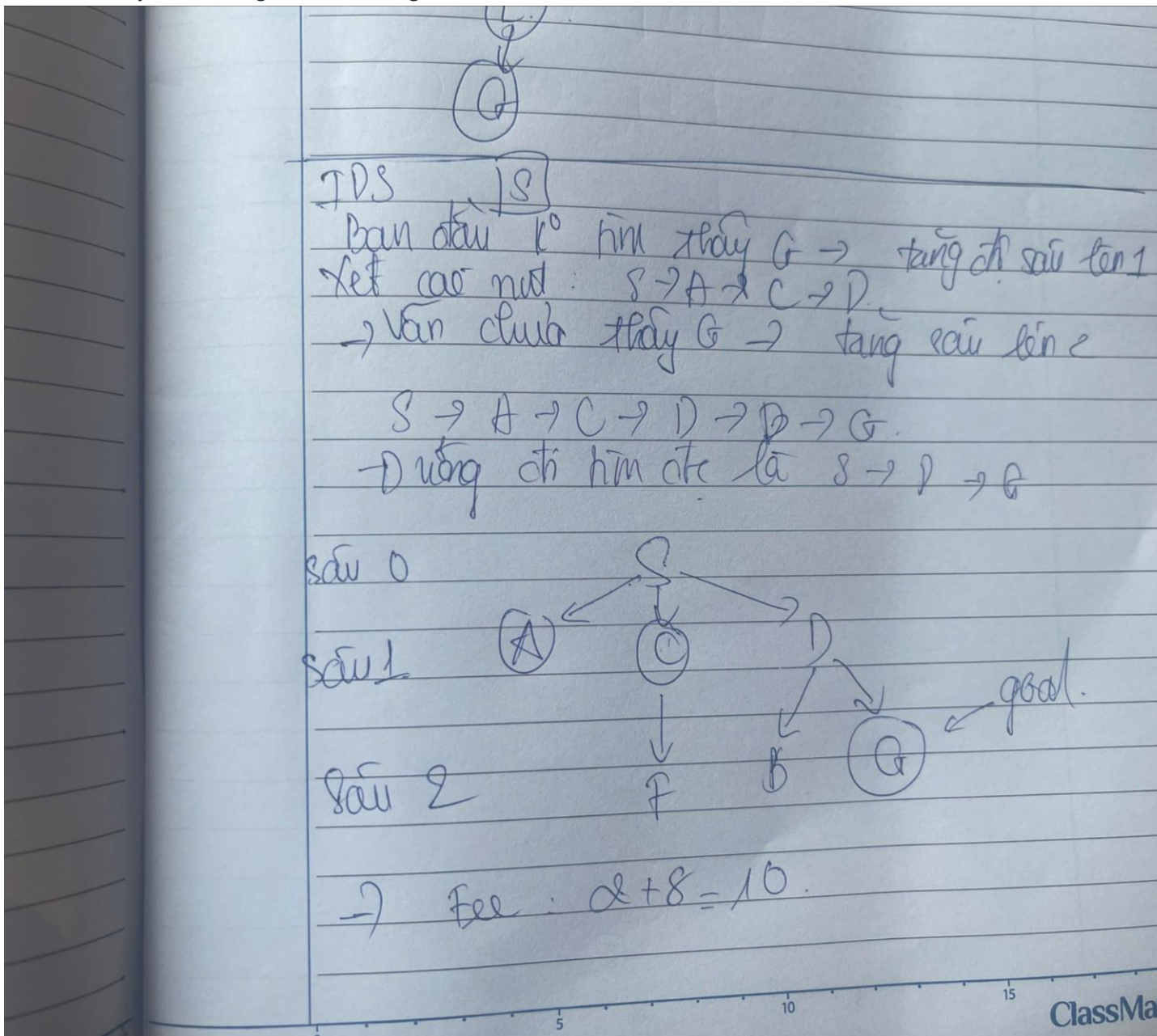
DFS

Ban đầu không duyệt các nút:

1.3. Iterative Deepening Search (IDS)

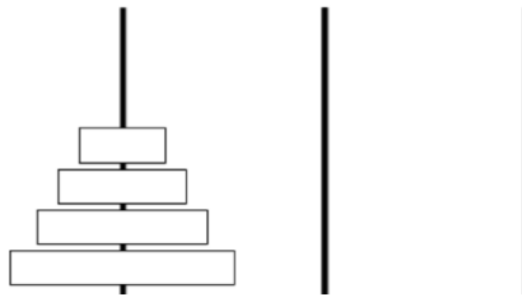
Giới hạn	Các nút được duyệt
0	S
1	S, A, C, D

IDS chạy DFS nhiều lần, lần lượt với giới hạn 0, 1 rồi 2. Giới hạn 0 chỉ xét S; giới hạn 1 chạy lại từ S để xét S, A, C, D. Đến giới hạn 2, nhánh $S \rightarrow C \rightarrow F$ dừng tại F vì đã đủ 2 cạnh; sau đó xét $S \rightarrow D \rightarrow B$ và $S \rightarrow D \rightarrow G$. Tìm thấy G nên dừng. Path cuối cùng: $S \rightarrow D \rightarrow G$; total cost = $2 + 8 = 10$.



BFS xét theo độ sâu tăng dần. Từ S, các nút mức 1 là A, C, D; ở mức 2 có B, F, G. Theo quy tắc phá hòa alphabet của đề, B được xét trước F rồi mới đến G. Vì đường $S \rightarrow D \rightarrow G$ chỉ có 2 cạnh, BFS trả về đường này, dù chi phí 10 không phải nhỏ nhất. DFS thì đi hết một nhánh trước: $S \rightarrow A$ (hết đường), quay lại $S \rightarrow C \rightarrow F \rightarrow E \rightarrow G$. DFS dùng stack/đệ quy, còn BFS duyệt lần lượt theo từng mức bằng queue (có phá hòa theo đề).

Bài 2. Tháp Hà Nội



2.1. Biểu diễn trạng thái

Biểu diễn trạng thái bằng $s = (p_1, p_2, \dots, p_n)$, trong đó $p_i \in \{1, 2, 3\}$ là số cọc đang chứa đĩa i ; đĩa 1 nhỏ nhất và đĩa n lớn nhất. Ví dụ với 4 đĩa, trạng thái $(2, 1, 1, 1)$ nghĩa là đĩa nhỏ nhất ở cọc 2, ba đĩa còn lại ở cọc 1. Trên mỗi cọc, đĩa nhỏ luôn nằm trên đĩa lớn nên không cần lưu thứ tự riêng.

2.2. Kích thước không gian trạng thái

Mỗi đĩa có thể nằm ở một trong 3 cọc. Vì có n đĩa nên số cách gán vị trí là $3 \times 3 \times \dots \times 3 = 3^n$. Với 4 đĩa như hình, số trạng thái là $3^4 = 81$. Mỗi cách gán đều tương ứng với một cách xếp hợp lệ khi các đĩa trên cùng cọc được xếp theo đúng kích thước.

2.3. Trạng thái bắt đầu

Với 4 đĩa, $S_0 = (1, 1, 1, 1)$. Tổng quát n đĩa: $S_0 = (1, 1, \dots, 1)$.

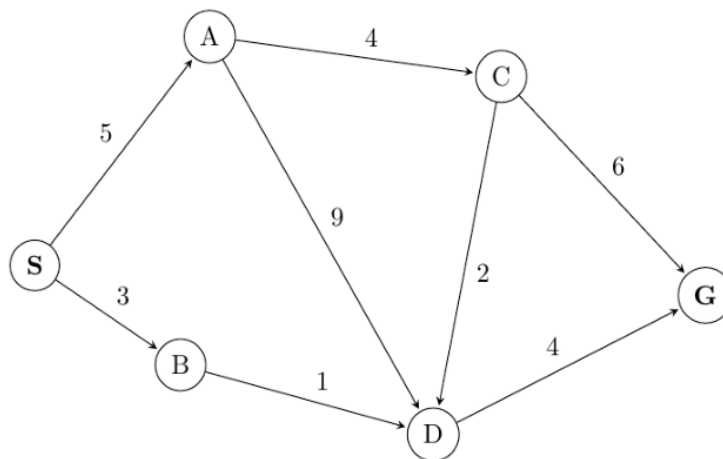
2.4. Các hành động hợp lệ

Một hành động là chuyển đúng 1 đĩa từ cọc i sang cọc j , $i \neq j$. Đĩa chuyển phải nằm trên cùng của cọc i . Cọc j phải rỗng hoặc đĩa đang ở trên cùng cọc j phải lớn hơn đĩa chuyển. Ví dụ từ $S_0 = (1, 1, 1, 1)$, chuyển đĩa 1 sang cọc 2 sẽ được $(2, 1, 1, 1)$; không thể chuyển đĩa 2 trước vì đang bị đĩa 1 nằm đè lên.

2.5. Goal Test

$\text{goalTest}(s)$ trả về True khi tất cả các đĩa đều ở cọc 3. Với 4 đĩa, trạng thái đích là $(3, 3, 3, 3)$.

Bài 3. Thiết kế và phân tích heuristic



3.1. Heuristic admissible và consistent

Nút	S	A	B	C	D	G
$h(n)$	7	6	5	6	4	0
$h^*(n)$	8	10	5	6	4	0

Trước hết tính chi phí ngắn nhất từ mỗi nút về G: $h^*(G)=0$; $h^*(D)=4$; $h^*(C)=\min(6, 2+4)=6$; $h^*(B)=1+4=5$; $h^*(A)=\min(4+6, 9+4)=10$; $h^*(S)=\min(5+10, 3+5)=8$. Đây là chi phí thật để đối chiếu với $h(n)$, không phải các giá trị cần đoán.

Bộ $h(n)$ ở bảng là admissible vì tại mọi nút đều có $h(n) \leq h^*(n)$. Để kiểm tra consistent, xét điều kiện $h(u) \leq c(u,v) + h(v)$ trên từng cạnh. Chẳng hạn $S \rightarrow B$: $7 \leq 3+5=8$; $B \rightarrow D$: $5 \leq 1+4=5$; $C \rightarrow D$: $6 \leq 2+4=6$; $D \rightarrow G$: $4 \leq 4+0=4$. Các cạnh còn lại cũng đều thỏa: $S \rightarrow A$ ($7 \leq 11$), $A \rightarrow C$ ($6 \leq 10$), $A \rightarrow D$ ($6 \leq 13$), $C \rightarrow G$ ($6 \leq 6$). Do đó bộ heuristic vừa admissible vừa consistent.

3.2. Chạy A*

Ở mỗi lượt, A* chọn nút trong frontier có $f(n)=g(n)+h(n)$ nhỏ nhất. Trong đó $g(n)$ là chi phí đi từ S đến n, còn $h(n)$ là ước lượng chi phí từ n tới G. Diễn biến cụ thể như bảng sau:

Nút lấy ra	Các nút trong frontier sau khi cập nhật (g; h; f)	Nút chọn tiếp
S	A (5; 6; 11), B (3; 5; 8)	B, vì $f=8$ nhỏ hơn 11
B	A (5; 6; 11), D (4; 4; 8)	D, vì $f=8$
D	A (5; 6; 11), G (8; 0; 8)	G, vì $f=8$
G	Đã gặp đích, dừng	—

Lần lượt lấy ra S, B, D rồi gặp G. Nếu kiểm tra đích trước khi thêm vào explored set: Explored set = {S, B, D}. Truy vết nút cha được path $S \rightarrow B \rightarrow D \rightarrow G$, total cost = $3 + 1 + 4 = 8$. Đây cũng là chi phí nhỏ nhất trên đồ thị.

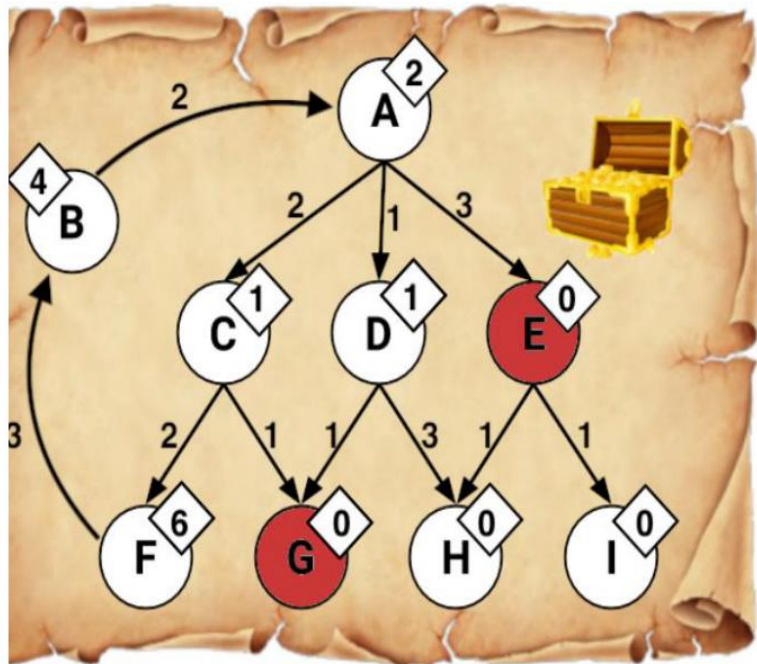
3.3. Admissible nhưng không consistent

Chọn bộ khác: $h(S)=8$, $h(A)=10$, $h(B)=0$, $h(C)=6$, $h(D)=4$, $h(G)=0$. Bộ này vẫn admissible vì từng giá trị không lớn hơn $h^*(n)$ trong bảng 3.1 (ví dụ $h(S)=8=h^*(S)$, còn $h(B)=0 \leq 5$). Nhưng cạnh $S \rightarrow B$ lại không consistent: cần có $h(S) \leq c(S,B)+h(B)$, trong khi $8 > 3+0=3$. Như vậy admissible chưa chắc consistent.

3.4. Vì sao consistent thì admissible?

Giả sử đường đi ngắn nhất từ n đến G là $n \rightarrow n_1 \rightarrow n_2 \rightarrow \dots \rightarrow G$. Do consistent, ta có $h(n) \leq c(n,n_1)+h(n_1)$; $h(n_1) \leq c(n_1,n_2)+h(n_2)$,... Cộng các bất đẳng thức này sẽ triệt tiêu những h ở giữa, suy ra $h(n) \leq$ tổng chi phí của đường đi ngắn nhất $+h(G)$. Vì $h(G)=0$ nên $h(n) \leq h^*(n)$. Đây chính là điều kiện admissible.

Bài 4. Treasure Hunting



A là điểm xuất phát; E và G là hai đảo có kho báu (đích). Khi gặp đích lúc lấy khỏi frontier thì dừng; nếu các giá trị ưu tiên bằng nhau, chọn theo alphabet. Các con số trên cạnh là chi phí thực, còn số trong hình thoi là $h(n)$.

Thuật toán	Explored set	Path returned	Cost
DFS	A, C, F, B	$A \rightarrow C \rightarrow G$	$2 + 1 = 3$
BFS	A, C, D	$A \rightarrow E$	3
UCS	A, D, C	$A \rightarrow D \rightarrow G$	$1 + 1 = 2$
Greedy	A	$A \rightarrow E$	3
A*	A, D	$A \rightarrow D \rightarrow G$	$1 + 1 = 2$

DFS: đi $A \rightarrow C \rightarrow F \rightarrow B$, thấy B chỉ quay về A đã xét nên quay lui; sau đó từ C chọn G. Path $A \rightarrow C \rightarrow G$, cost = $2+1=3$. BFS: xét các nút độ sâu 1 là C, D, E; E là đích nên chọn $A \rightarrow E$, cost = 3. UCS: sau A thì D có $g=1$, C có $g=2$, E có $g=3$; từ D sinh G có $g=2$. C và G cùng $g=2$ nên ưu tiên C theo alphabet, sau đó chọn G; cost = $1+1=2$. Greedy chỉ nhìn h: từ A, $h(E)=0$ nhỏ nhất nên chọn $A \rightarrow E$, cost = 3. A*: từ A có $f(C)=2+1=3$, $f(D)=1+1=2$, $f(E)=3+0=3$; chọn D. Từ D sinh G với $f(G)=2+0=2$ nên chọn G; cost = 2.

Bài 5. Câu hỏi Đúng/Sai

Câu 1. DFS luôn mở rộng ít nhất nhiều node bằng A* dùng admissible heuristic.

Sai. DFS có thể may mắn gặp đích ngay trên nhánh được thử đầu tiên nên mở rộng rất ít nút. Trong khi đó A* vẫn có thể phải xét các nút khác có $f(n)$ thấp. Vì phụ thuộc vào đồ thị và thứ tự duyệt, không thể khẳng định DFS luôn mở rộng ít nhất nhiều nút bằng A*.

Câu 2. Manhattan distance là admissible heuristic cho số nước đi ít nhất của quân xe.

Sai. Manhattan distance đếm số ô lệch theo hàng/cột, nhưng quân xe có thể đi nhiều ô chỉ trong một nước. Ví dụ hai ô cùng hàng cách 7 ô, đường không bị chắn: quân xe đi 1 nước, trong khi Manhattan distance = 7 > 1, nên không admissible.

Câu 3. Euclidean distance là admissible heuristic cho bài toán tìm đường Pacman.

Đúng trong thiết lập Pacman thông thường: mỗi bước sang một ô kề theo bốn hướng, chi phí mỗi bước bằng 1. Đường thẳng Euclidean giữa hai ô không dài hơn bất kỳ đường đi hợp lệ nào, đặc biệt khi có tường phải đi vòng. Vì vậy nó không ước lượng vượt chi phí thật.

Câu 4. Tổng của nhiều admissible heuristic vẫn luôn admissible.

Sai. Hai heuristic riêng lẻ có thể đều không vượt chi phí thật nhưng tổng lại vượt. Ví dụ chi phí tối ưu còn lại $h^*=5$, chọn $h_1=4$ và $h_2=4$. Cả hai đều admissible vì $4 \leq 5$, nhưng $h_1+h_2=8 > 5$ nên tổng không admissible.

Câu 5. Admissibility kéo theo consistency.

Sai. Admissible chỉ yêu cầu $h(n) \leq h^*(n)$, còn consistent yêu cầu điều kiện trên từng cạnh. Trong Bài 3, bộ $h(S)=8, h(B)=0$ (các nút khác như ở mục 3.3) vẫn không vượt h^* , nhưng cạnh $S \rightarrow B$ có $8 > 3+0=3$. Do đó admissible không kéo theo consistent.

Câu 6. A* graph search luôn tối ưu nếu h admissible.

Sai nếu A* graph search không cho mở lại nút đã đóng. Xét $S \rightarrow A$ (2), $S \rightarrow B$ (1), $B \rightarrow A$ (0,5), $A \rightarrow G$ (2) với $h(S)=0, h(A)=0, h(B)=2,5, h(G)=0$ (đều admissible). Ban đầu $f(A)=2, f(B)=3,5$ nên A bị đóng với $g(A)=2$; từ A sinh G có $g=4$. Sau đó xét B mới phát hiện đường $S \rightarrow B \rightarrow A$ có $g(A)=1,5$. Nếu A không được mở lại, thuật toán vẫn trả đường $S \rightarrow A \rightarrow G$, cost 4, trong khi đường tối ưu $S \rightarrow B \rightarrow A \rightarrow G$ chỉ tốn $1+0,5+2=3,5$. Nếu cho phép mở lại nút đã đóng khi tìm được g tốt hơn, A* có thể khắc phục trường hợp này.